Counter

NAMA: Rahadyan Rakha Dzakwan Putra

KELAS: 1 D4 Teknik Komputer B

NRP: 3223600059

DOSEN: Reni Soelistijorini B.Eng, MT

MATA KULIAH: Rangkaian Logika

TANGGAL: 23 Mei 2024

ASISTEN DOSEN:.....

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2023

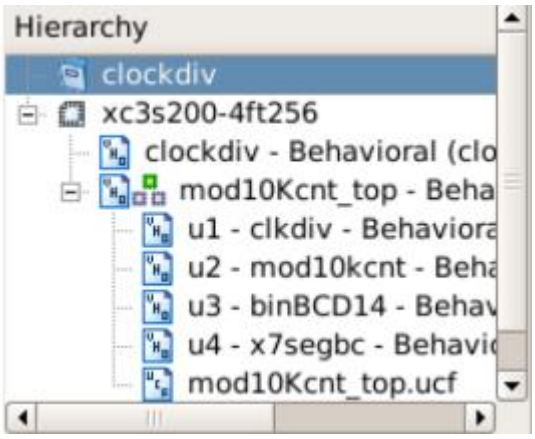
Tujuan:

1. Dapat menjelaskan rangkaian pembagi clock.
2. Dapat membuat rangkaian Counter modulus 10k.

Alat dan Bahan:

1. PC beserta software ISE Design Suite 13.1
2. Xilinx Spartan 3
3. Downloader JTAG USB
4. Adaptor 5V

Percobaan:



a. Program clockdiv.vhdl (clock devider)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity clockdiv is

        port ( Clk25Mhz: in STD_LOGIC;
        Clk: out STD_LOGIC);

end clockdiv;

architecture Behavioral of clockdiv is

        constant max: integer := 25000;
        constant half : integer := max/2;
        signal count : integer range 0 to max;

begin
        process
        begin
                wait until Clk25Mhz' EVENT and Clk25Mhz = '1' ;
                if count < max then
                        count <= count + 1;
                else
                        count <= 0;
                end if;
                if count < half then
                        Clk <= '0' ;
                else
                        Clk <= '1' ;
                end if;
end process;

end Behavioral;
```

b. Program clkdiv.vhdl (Clock devider 48Hz dan 190Hz)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_unsigned.ALL;

entity clkdiv is
	Port ( mclk : in STD_LOGIC;
	clr : in STD_LOGIC;
	clk190 : out STD_LOGIC;
	clk48 : out STD_LOGIC;
	q_out : out std_logic_vector(23 downto 0));
end clkdiv;

architecture Behavioral of clkdiv is
	signal q:std_logic_vector (23 downto 0);

begin
	process(mclk, clr )
	begin
	if clr ='1' then
		q <= x"000000";
	elsif mclk' event and mclk ='1' then
		q <= q + 1;
	end if;

	end process;

	clk48 <= q(19);
	clk190 <= q(17);
	q_out <= std_logic_vector(q);

end Behavioral;
```

b. Program mod10kcnt.vhdl (Counter modulus 10K)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_unsigned.ALL;

entity mod10kcnt is
	Port ( clr : in STD_LOGIC;
		clk : in STD_LOGIC;
		q : out STD_LOGIC_VECTOR (13 downto 0));
end mod10kcnt;

architecture Behavioral of mod10kcnt is
signal count :STD_LOGIC_VECTOR (13 downto 0);
begin
	process(clk,clr )
	begin
	if clr = '1' then
		count <= (others => '0' );
	elsif clk' event and clk ='1' then
	if conv_integer (count) = 9999 then
		count <= (others => '0');
	else
		count <= count + 1;
	end if;
	end if;
	end process;
	q <= count ;

end Behavioral;
```

c. Program binBCD14.vhdl (Biner 14 bit to BCD)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_unsigned.ALL;

entity binBCD14 is
	Port ( b : in STD_LOGIC_VECTOR (13 downto 0);
		p : out STD_LOGIC_VECTOR (16 downto 0));
end binBCD14;

architecture Behavioral of binBCD14 is

begin
	process(b)
	variable z : STD_LOGIC_VECTOR (32 downto 0);
	begin
		for i in 0 to 32 loop
		z(i) := '0' ;
		end loop;
		z(16 downto 3) := b;
		for i in 0 to 10 loop
			if z(17 downto 14) > 4 then
				z(17 downto 14) := z(17 downto 14) + 3;
			end if;
			if z(21 downto 18) > 4 then
				z(21 downto 18) := z(21 downto 18) + 3;
			end if;
			if z(25 downto 22) > 4 then
				z(25 downto 22) := z(25 downto 22) + 3;
			end if;
			if z(29 downto 26) > 4 then
				z(29 downto 26) := z(29 downto 26) + 3;
			end if;
			z(32 downto 1) := z(31 downto 0);
		  end loop;
		  p <= z(30 downto 14);
	end process;

end Behavioral;
```

d. Program x7segbc.vhdl (x7segbc)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_unsigned.ALL;

entity x7segbc is
	port( x : in std_logic_vector (15 downto 0);
	cclk : in std_logic;
	clr : in std_logic;
```

```
        a_to_g : out std_logic_vector (6 downto 0);
        an : out std_logic_vector (3 downto 0);
        dp : out std_logic
);
end x7segbc;

        architecture Behavioral of x7segbc is
        signal s : std_logic_vector (1 downto 0);
        signal digit : std_logic_vector (3 downto 0);
        signal aen : std_logic_vector (3 downto 0);
begin
        dp <= '1';
        -- set aen(3 downto 0) for leading blanks
        aen(3) <= x(15) or x(14) or x(13) or x(12);
        aen(2) <= x(15) or x(14) or x(13) or x(12)
        or x(11) or x(10) or x(9) or x(8);
        aen(1) <= x(15) or x(14) or x(13) or x(12)
        or x(11) or x(10) or x(9) or x(8)
        or x(7) or x(6) or x(5) or x(4);

        aen(0) <= '1';
        -- Quad 4-to-1 MUX: mux44
        process(s,x)
        begin
        case s is
        when "00" => digit <= x(3 downto 0);
        when "01" => digit <= x(7 downto 4);
        when "10" => digit <= x(11 downto 8);
        when others => digit <= x(15 downto 12);
        end case;
        end process;

        -- 7- segment decoder : hex7seg
        process(digit)
        begin
            case digit is
                when x"0" => a_to_g <= "0000001" ; -- 0
                when x"1" => a_to_g <= "1001111" ; -- 1
                when x"2" => a_to_g <= "0010010" ; -- 2
                when x"3" => a_to_g <= "0000110" ; -- 3
                when x"4" => a_to_g <= "1001100" ; -- 4
                when x"5" => a_to_g <= "0100100" ; -- 5
                when x"6" => a_to_g <= "0100000" ; -- 6
                when x"7" => a_to_g <= "0001111" ; -- 7
                when x"8" => a_to_g <= "0000000" ; -- 8
                when x"9" => a_to_g <= "0000100" ; -- 9
                when x"A" => a_to_g <= "0001000" ; -- a
                when x"B" => a_to_g <= "1100000" ; -- b
                when x"C" => a_to_g <= "0110001" ; -- c
                when x"D" => a_to_g <= "1000010" ; -- d
                when x"E" => a_to_g <= "0110000" ; -- e
                when others => a_to_g <= "0111000" ; -- f
            end case;
```

```
-- 2 bit counter
process(cclk, clr )
begin
        if clr ='1' then
        s <= "00" ;
        elsif cclk' event and cclk='1' then
        s <= s + 1;
  end if;
end process;
```

e. Program mod10kcnt_top.vhdl (Mod10Kcnt_to

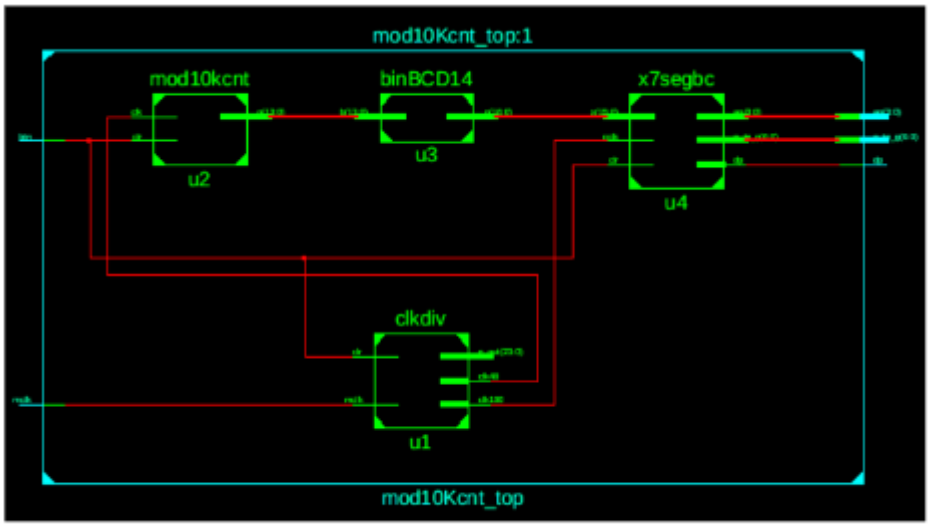
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity mod10Kcnt_top is
	port( mclk : in std_logic;
		btn : in std_logic;
		a_to_g : out std_logic_vector (6 downto 0);
		an : out std_logic_vector (3 downto 0);
		dp : out std_logic);

end mod10Kcnt_top;
architecture Behavioral of mod10Kcnt_top is
component clkdiv is
	port( mclk : in std_logic;
		clr : in std_logic;
		clk190: out std_logic;
		clk48: out std_logic
		);
end component;
component mod10kcnt is
	port( clr : in STD_LOGIC;
		clk : in STD_LOGIC;
		q : out STD_LOGIC_VECTOR (13 downto 0)
		);
end component;
component binBCD14 is
Port ( b : in STD_LOGIC_VECTOR (13 downto 0);
p : out STD_LOGIC_VECTOR (16 downto 0));

end component;
component x7segbc is
	port( x : in std_logic_vector (15 downto 0);
		cclk : in std_logic;
		clr : in std_logic;
		a_to_g : out std_logic_vector (6 downto 0);
		an : out std_logic_vector (3 downto 0);
		dp : out std_logic
		);
end component;
signal b : std_logic_vector (13 downto 0);
signal p : std_logic_vector (16 downto 0);
signal clr , clk48, clk190 : std_logic;
begin
	clr <= btn;
	u1: clkdiv
		port map(
```

```
        mclk => mclk,
        clr => clr,
        clk190 => clk190,
        clk48 => clk48
        );
        u2: mod10kcnt
                port map(
                clr => clr,
                clk => clk48,
                q => b
                );
        u3: binbcd14
                port map(
                b => b,
                p => p
                );
                u4: x7segbc
        port map(
                x => p(15 downto 0),
                cclk => clk190,
                clr => clr,
                a_to_g => a_to_g,
                an => an,
                dp => dp
                );
        end Behavioral;
```

f. RTL Schematic



g. Program mod10kcnt_top.ucf (konfigurasi pin)

```
NET mclk LOC = T9;
NET btn LOC = M13;
NET "a_to_g[0]" LOC = N16;
NET "a_to_g[1]" LOC = F13;
NET "a_to_g[2]" LOC = R16;
NET "a_to_g[3]" LOC = P15;
NET "a_to_g[4]" LOC = N15;
NET "a_to_g[5]" LOC = G13;
NET "a_to_g[6]" LOC = E14;
NET "an[0]" LOC = D14;
NET "an[1]" LOC = G14;
NET "an[2]" LOC = F14;
NET "an[3]" LOC = E13;
NET "dp" LOC = P16;
```

I. Output spartan 3

Ketika di reset counter akan mengulang dari 0:

J. Analisa

Dalam percobaan ini, berbagai program VHDL dikembangkan dan diimplementasikan pada FPGA Xilinx Spartan 3. Program clockdiv.vhdl menghasilkan pembagi clock yang menurunkan frekuensi dari 25 MHz menjadi 1 Hz, sedangkan clkdiv.vhdl menghasilkan clock dengan frekuensi 190 Hz dan 48 Hz. Program mod10kcnt.vhdl berfungsi sebagai counter modulus 10K yang menghitung dari 0 hingga 9999, kemudian kembali ke 0 setelah mencapai nilai maksimum. Program binBCD14.vhdl mengonversi bilangan biner 14-bit menjadi BCD, sedangkan x7segbc.vhdl mengendalikan tampilan 7-segmen menggunakan teknik multiplexing dan pembagi clock. Modul utama, mod10kcnt_top.vhdl, mengintegrasikan semua modul ini dan mengatur port ke pin FPGA, dengan konfigurasi pin di mod10kcnt_top.ucf. Hasilnya, perangkat Spartan 3 menampilkan hitungan yang terus bertambah dari 0 hingga 9999, lalu direset kembali ke 0.